

Programação Para Dispositivos Móveis II

PRIMEIRA APLICAÇÃO iOS

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

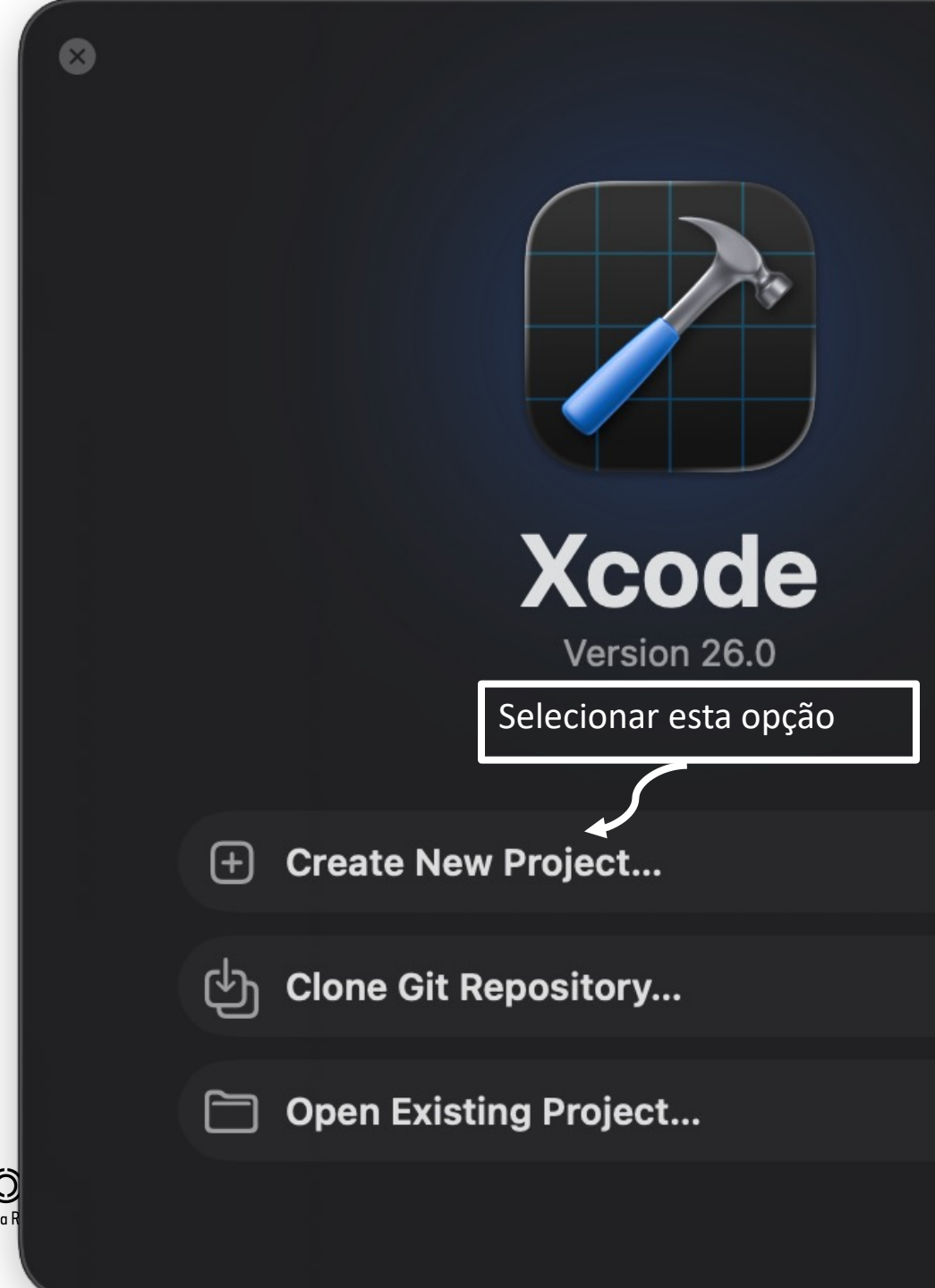
José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jmr@estg.ipp.pt, Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt

Hands on

Criar primeira app em iOS

1. Crie um novo projeto Xcode (Create a new Xcode Project);



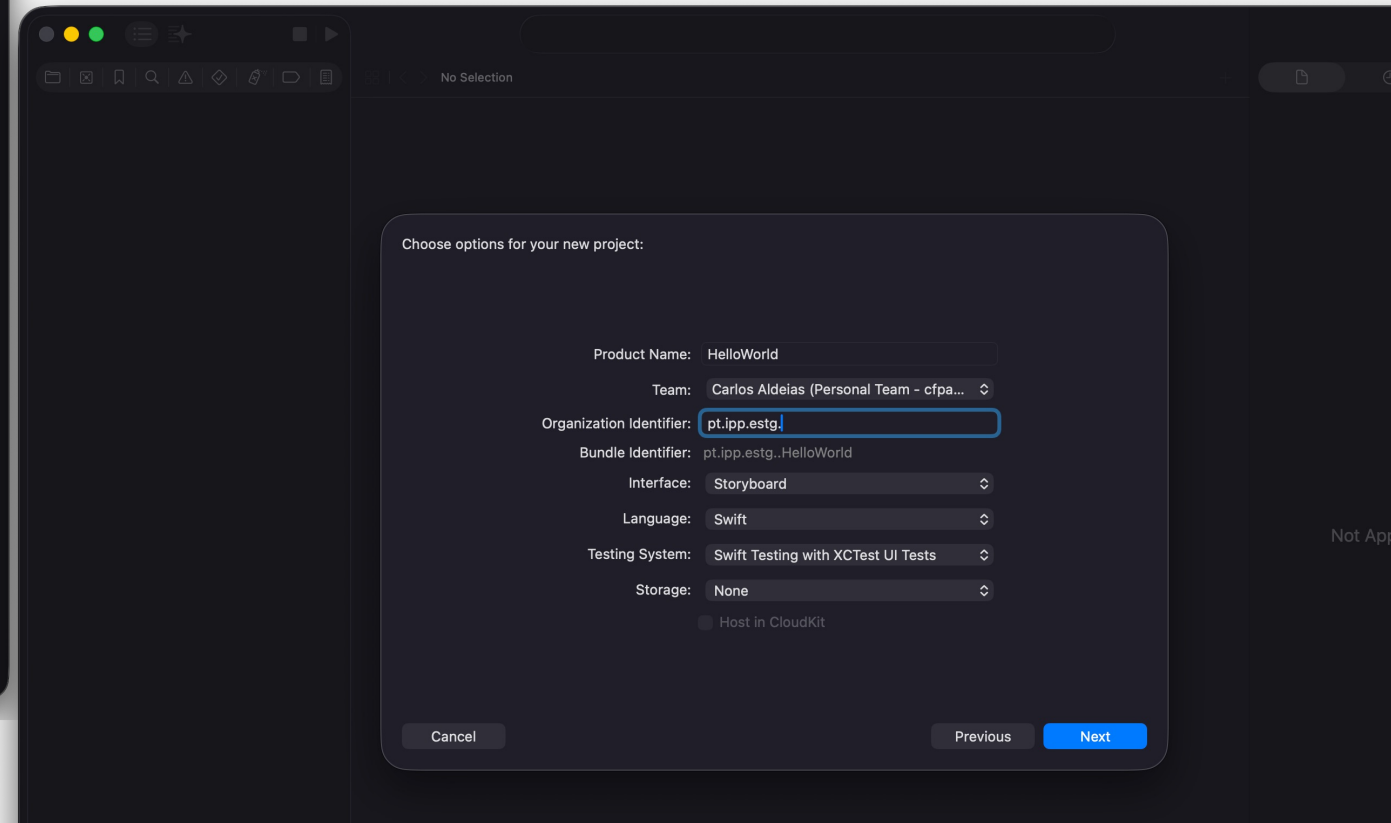
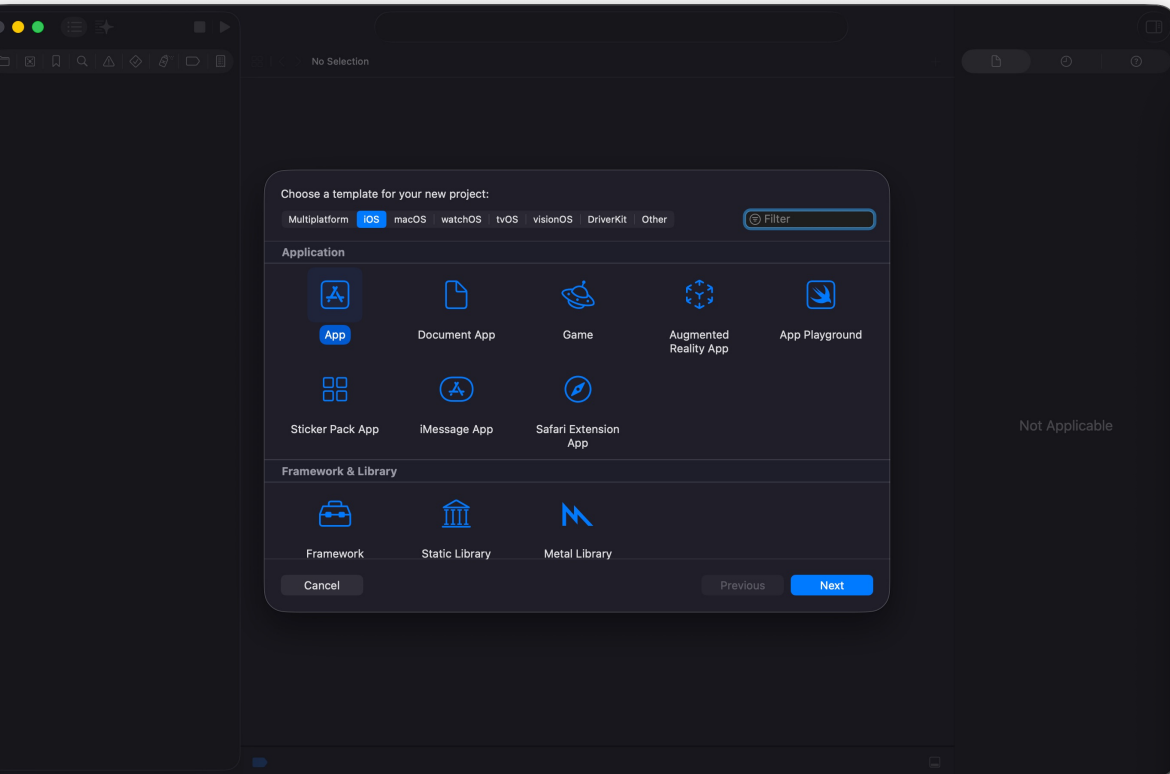
Hands-on

Criar primeira app em iOS

2. Selecione App dentro do menu iOS

3. Nome da aplicação: HelloWorld

4. Tipo de interface: storyboard



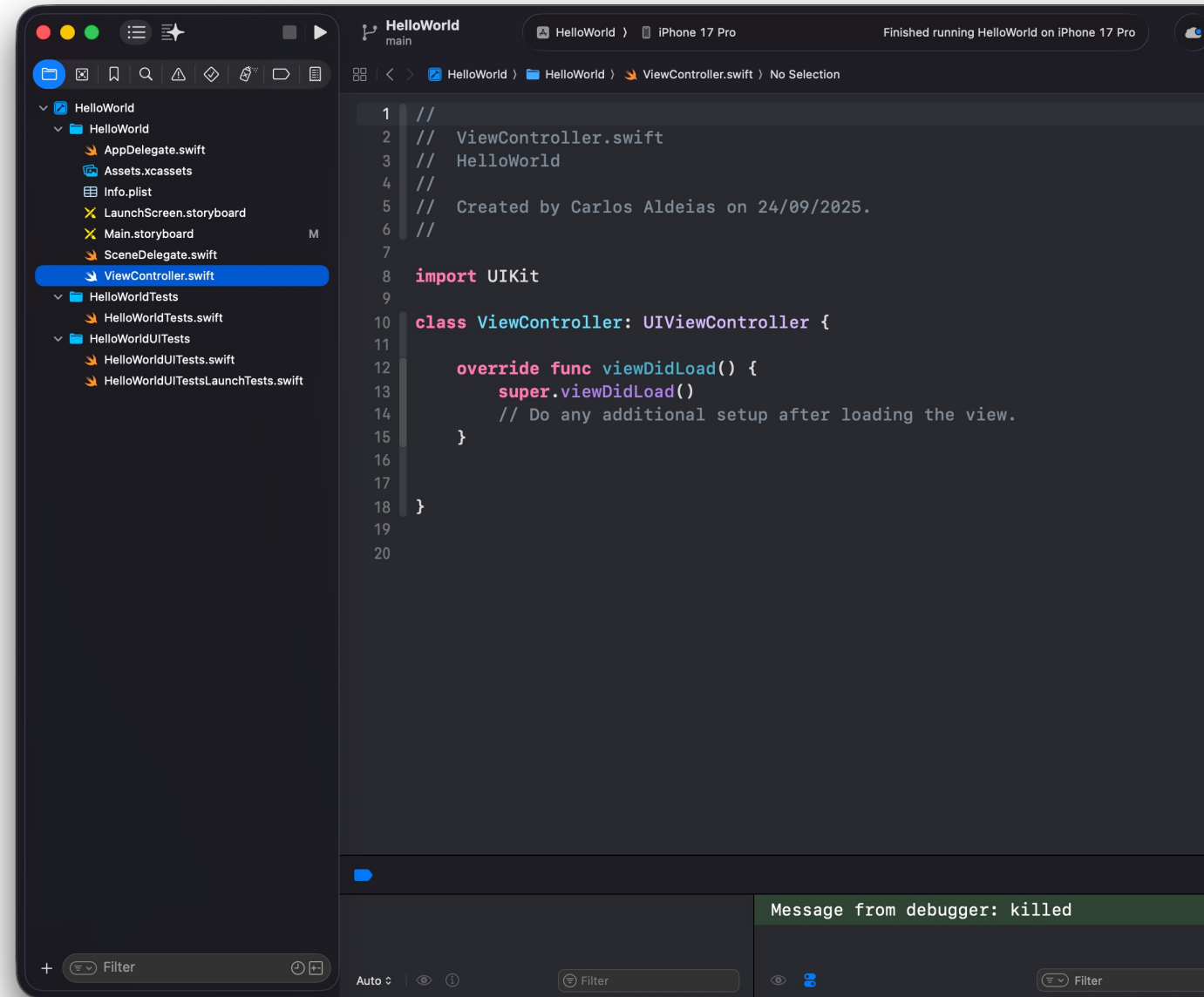
Hands-on

Criar primeira app em iOS

Criação de uma aplicação simples

- Contém um botão;
- No ecrã surge um nome previamente definido quando o botão é premido.

1. Abra o ficheiro `ViewController.swift`
2. O editor apresenta o código Swift que define uma classe `ViewController`

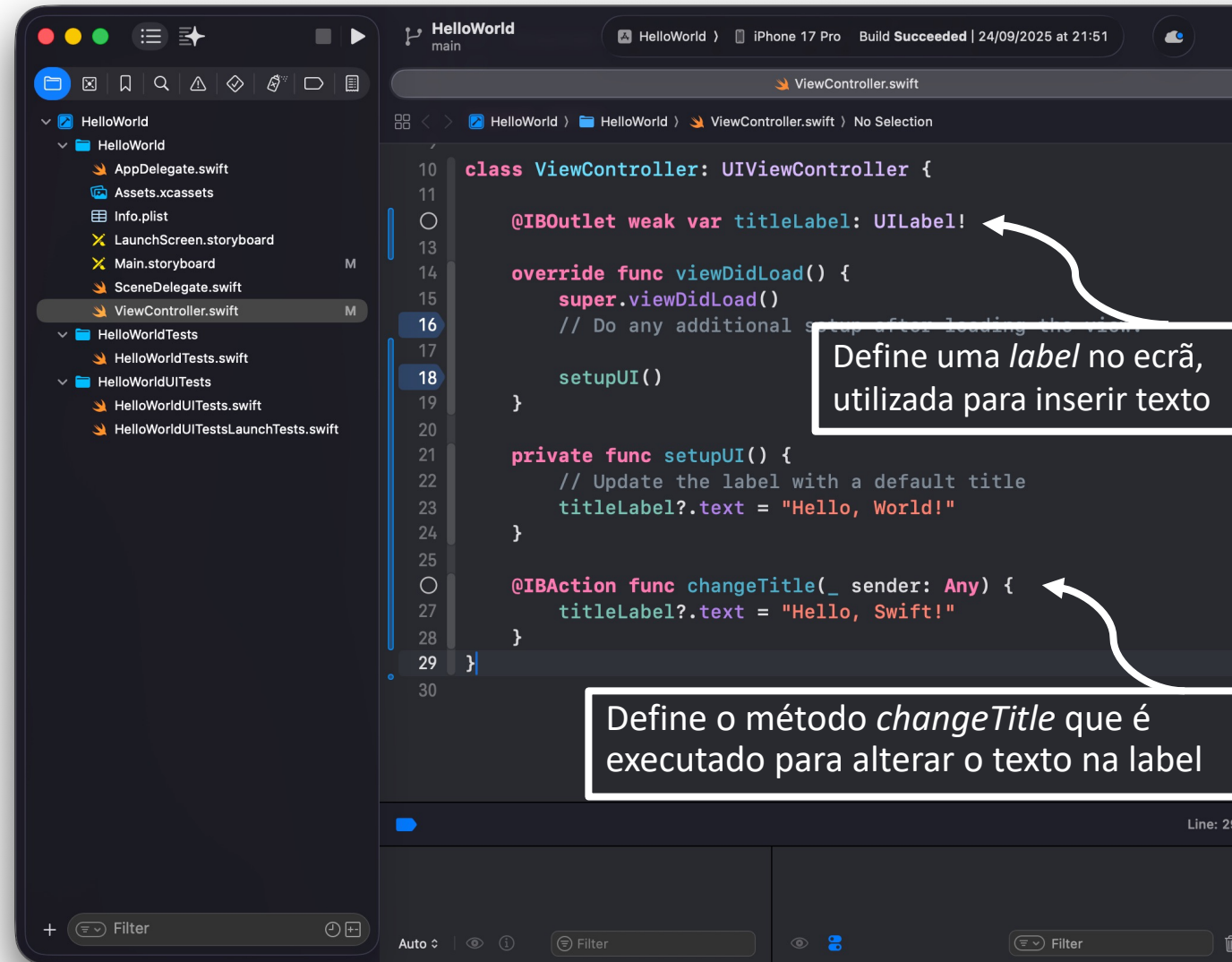


Hands-on

Criar primeira app em iOS

Existe uma função por omissão no código: `viewDidLoad`

- É executada imediatamente após a view ser carregada e possa ser utilizada para configurar a view
- É um bom local para se inserir código para configurar
 - Labels
 - Botões
 - Cores
 - Entre outros



Hands-on

Criar primeira app em iOS

Antes da declaração da *label* e do método foram adicionados os atributos

- @IBOutlet
 - permite controlar o objeto da interface (que aparece no ecrã) com código
- @IBAction
 - permite executar código quando uma determinada ação ocorre na interface como, por exemplo, tocar num botão
- Atributos necessários para conectar os objetos com o *interface designer*

O acrónimo IB significa *Interface Builder*

- Ferramenta utilizada pela Apple para criar interfaces

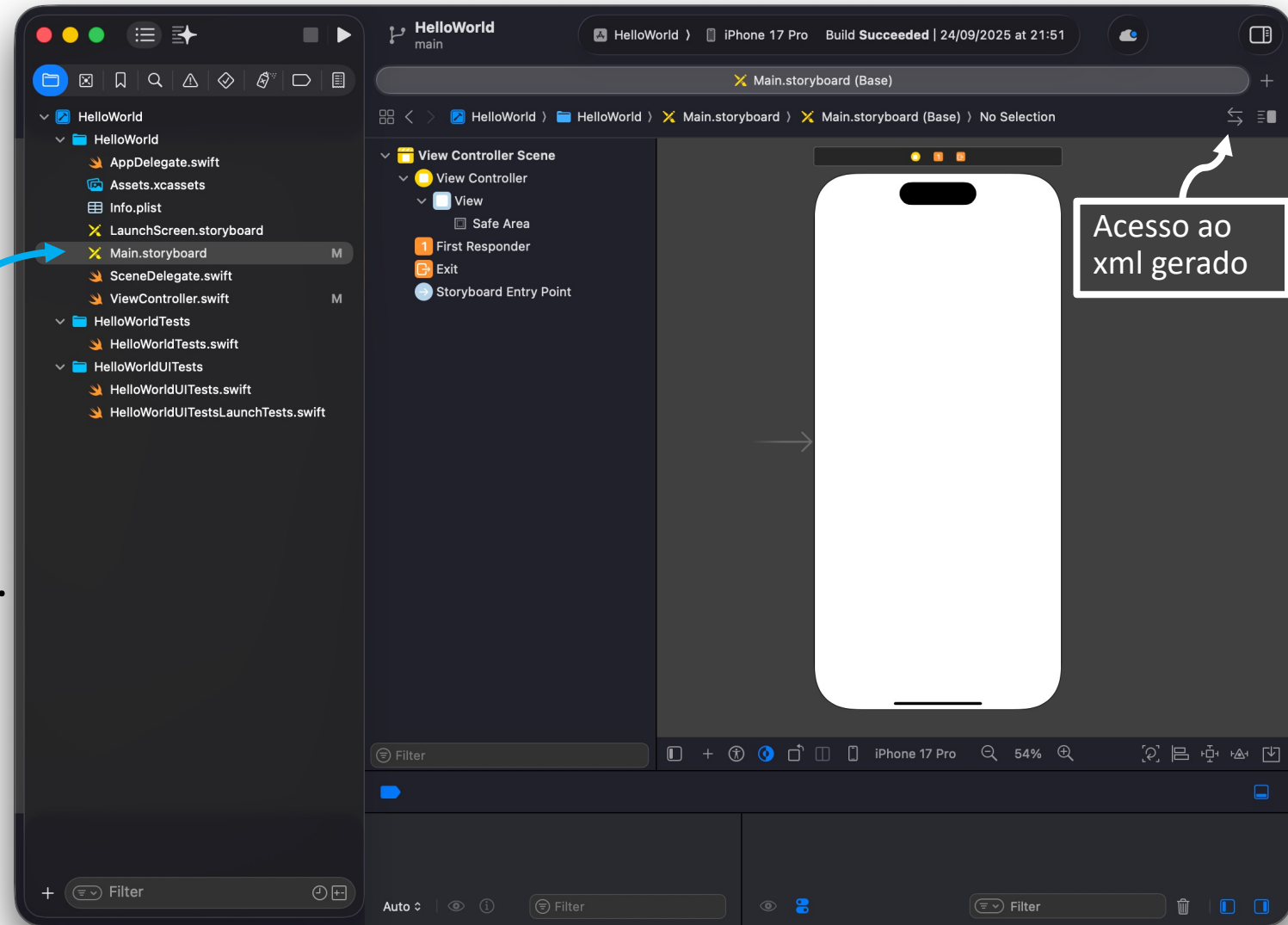
```
10 class ViewController: UIViewController {
11
12     @IBOutlet weak var titleLabel: UILabel!
13
14     override func viewDidLoad() {
15         super.viewDidLoad()
16         // Do any additional setup after loading the view.
17
18         setupUI()
19     }
20
21     private func setupUI() {
22         // Update the label with a default title
23         titleLabel?.text = "Hello, World!"
24     }
25
26     @IBAction func changeTitle(_ sender: Any) {
27         titleLabel?.text = "Hello, Swift!"
28     }
29 }
30
```

Hands-on

Criar primeira app em iOS

Para editar a interface do iPhone é necessário clicar no ficheiro `Main.storyboard`

- Um ficheiro do tipo storyboard contém toda a informação sobre uma única janela ou view. O Xcode também suporta ficheiros `.xib` (pronuncia-se zib)



Hands-on

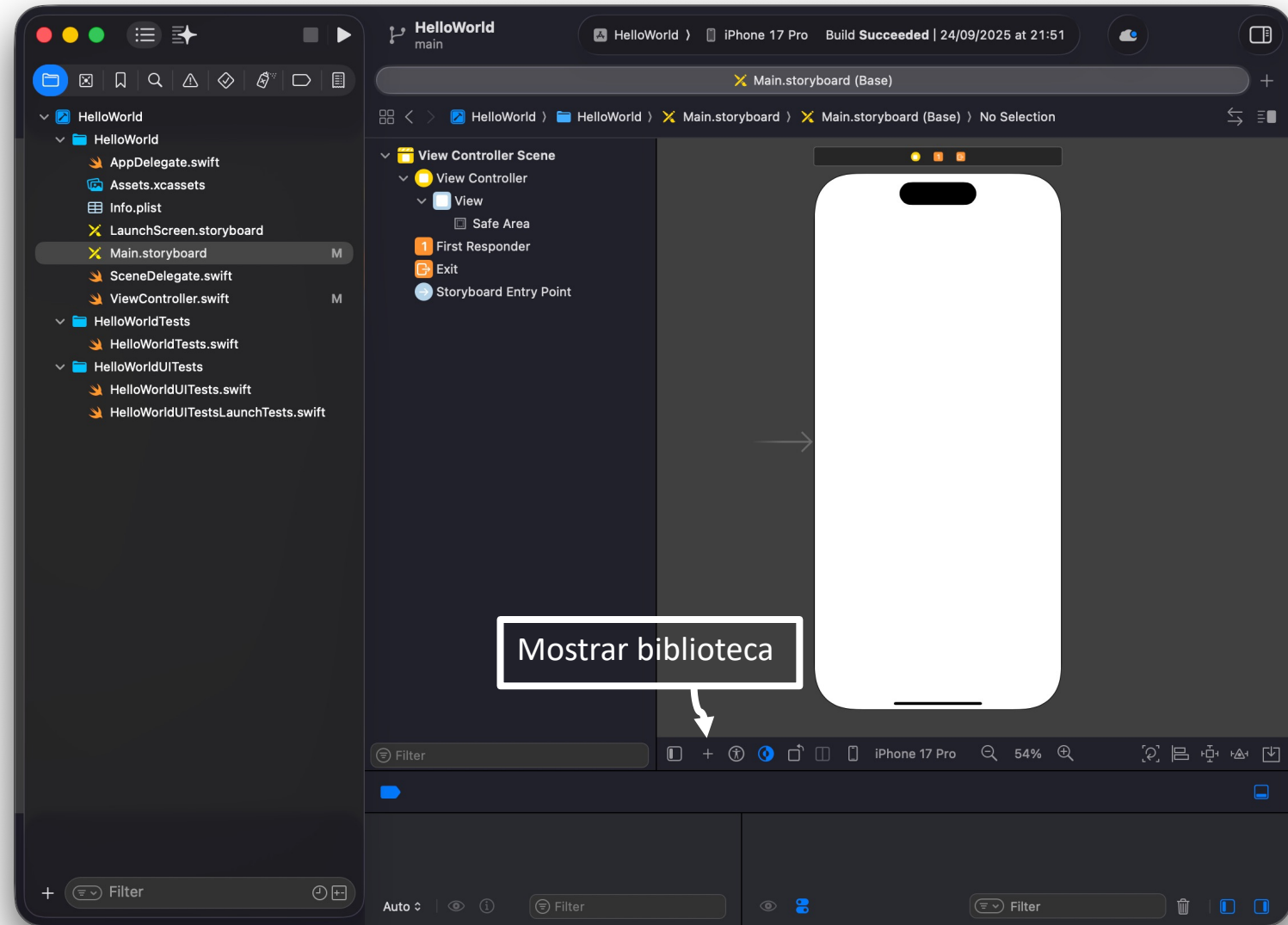
Criar primeira app em iOS

Cada ficheiro .xib representa um ecrã do iPhone ou iPad

- Aplicações com múltiplas views contêm vários ficheiros .xib;
- Múltiplas views podem ser armazenadas num ficheiro storyboard.

Vamos utilizar a interface do editor para conectar um objeto UI, como uma *label*, ao código (ViewController)

- Fácil como clicar e arrastar!



Hands-on

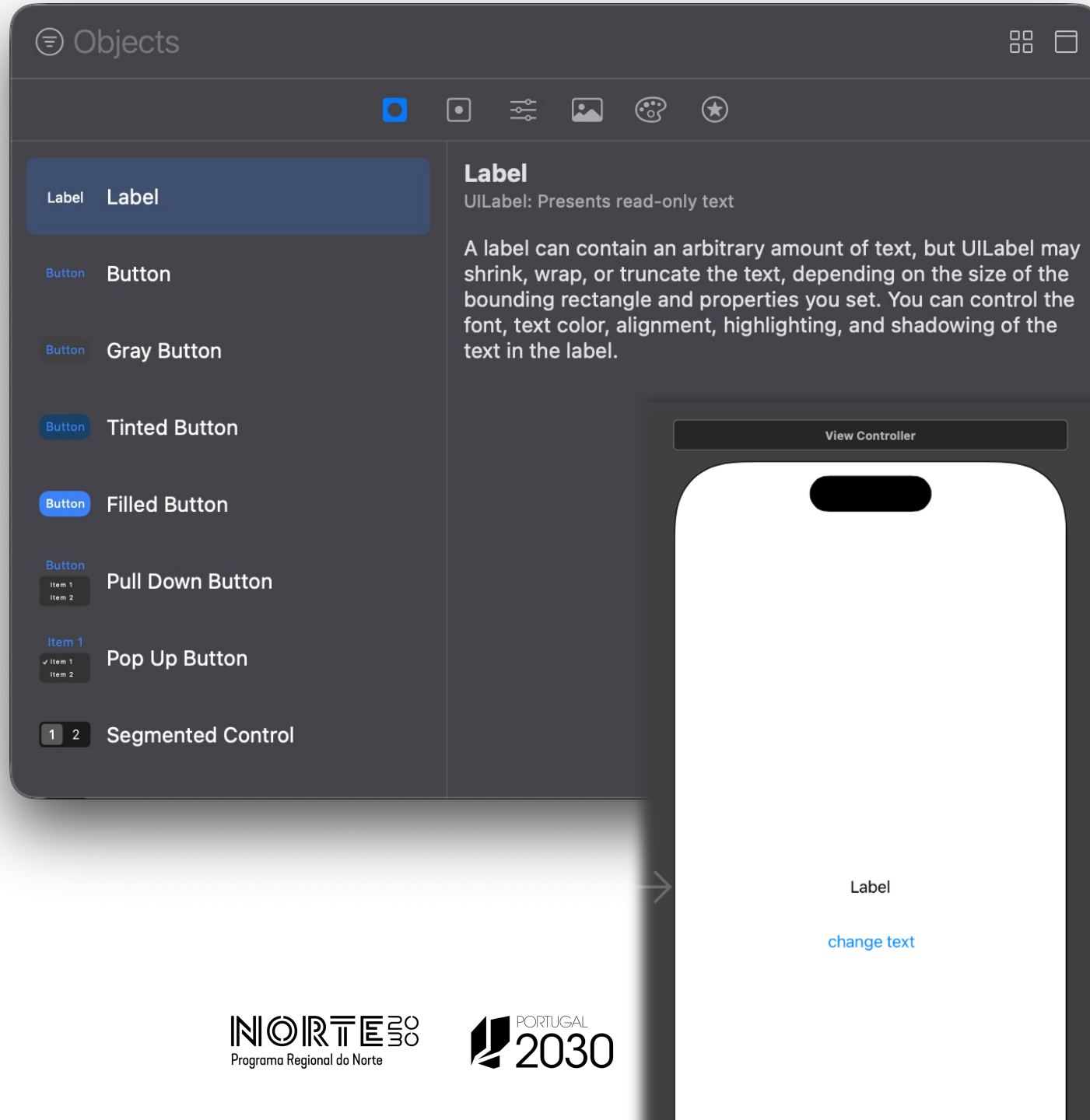
Criar primeira app em iOS

Clique no botão da biblioteca na barra de ferramentas

- Em versões mais antigas do Xcode tem o aspeto de um quadrado dentro de um círculo
- Versão mais recente tem o símbolo +

Procure na biblioteca o objeto pretendido e arraste para a view do iPhone

- Insira uma **Label**
- Insira um **Button**

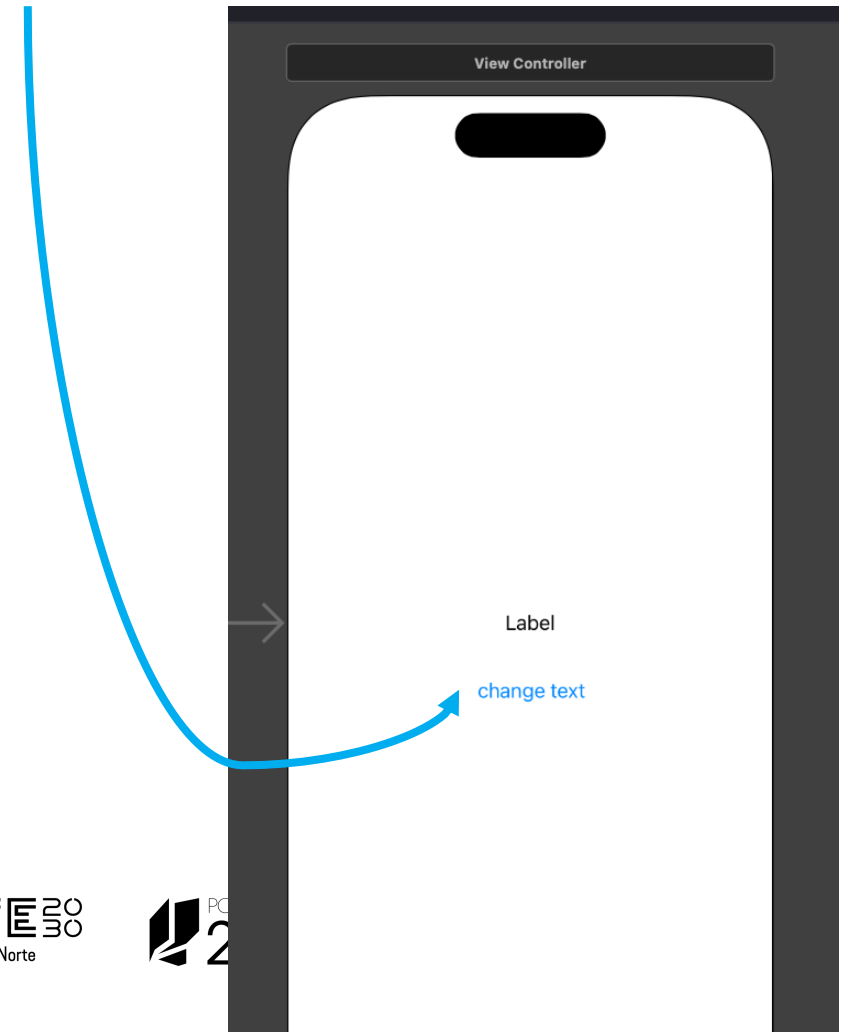


Hands-on

Criar primeira app em iOS

Faça duplo-clique no botão recentemente adicionado e altere o título para "*Change text*"

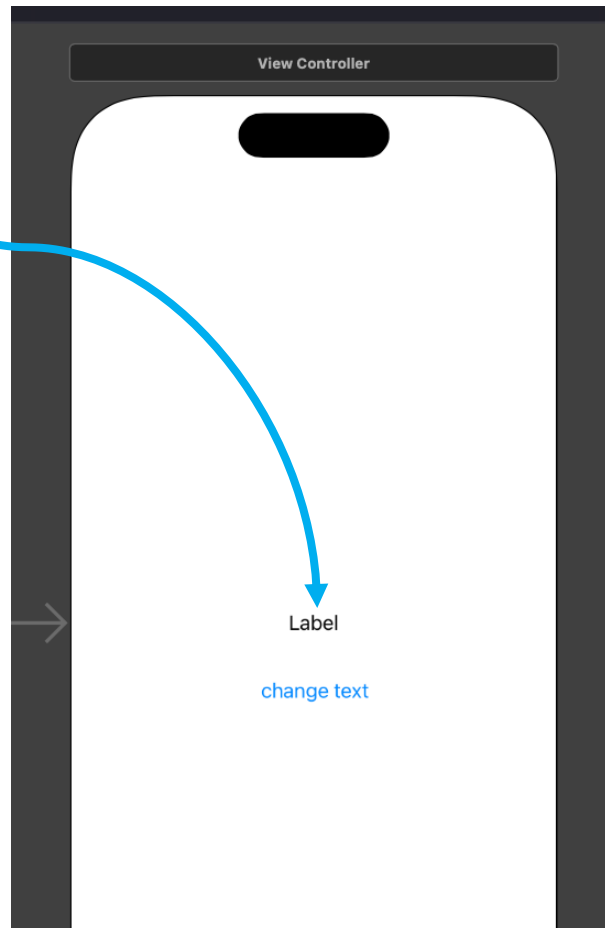
- Nota: Esta ação pode ser realizada através de código...



Hands-on

Criar primeira app em iOS

Vamos deixar o texto Label no objeto Label para ser fácil de encontrar. Caso se altere o texto para " " (string vazia) o objeto permaneceria, mas seria mais difícil de encontrar...

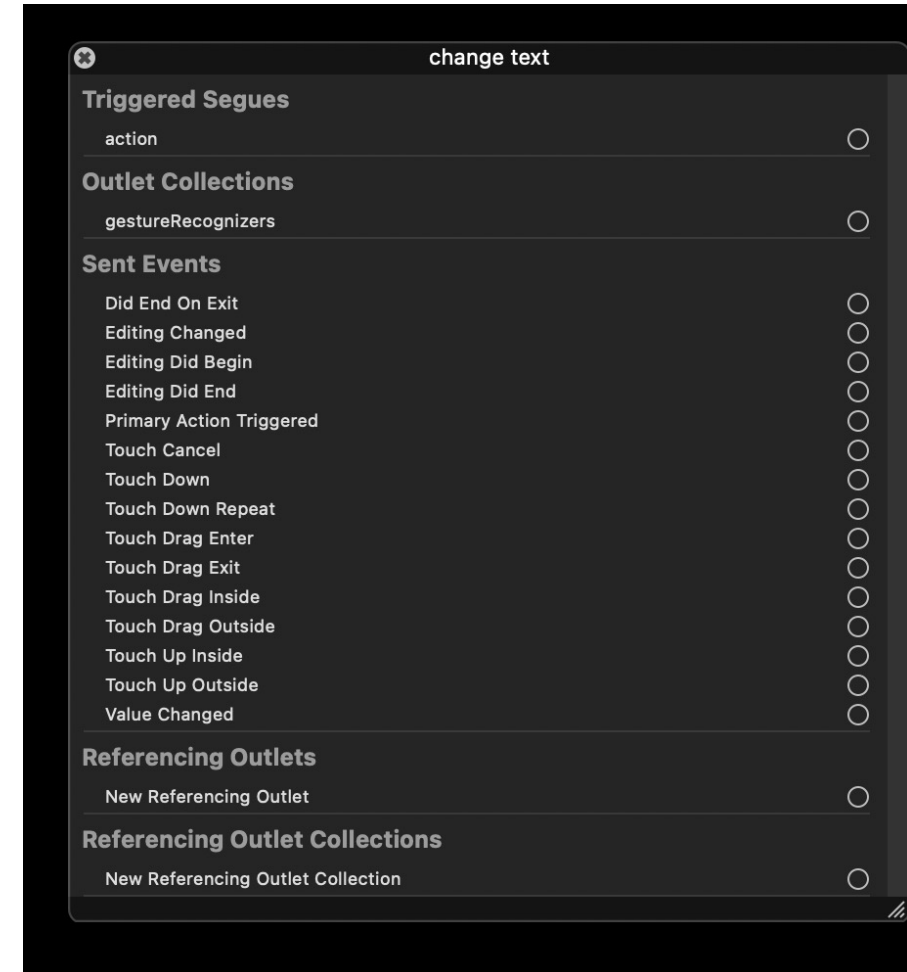


Hands-on

Criar primeira app em iOS

Nesta fase é possível conectar os objetos visuais ao código

- Clique com o botão direito no Button inserido, de modo a abrir o menu de conexão

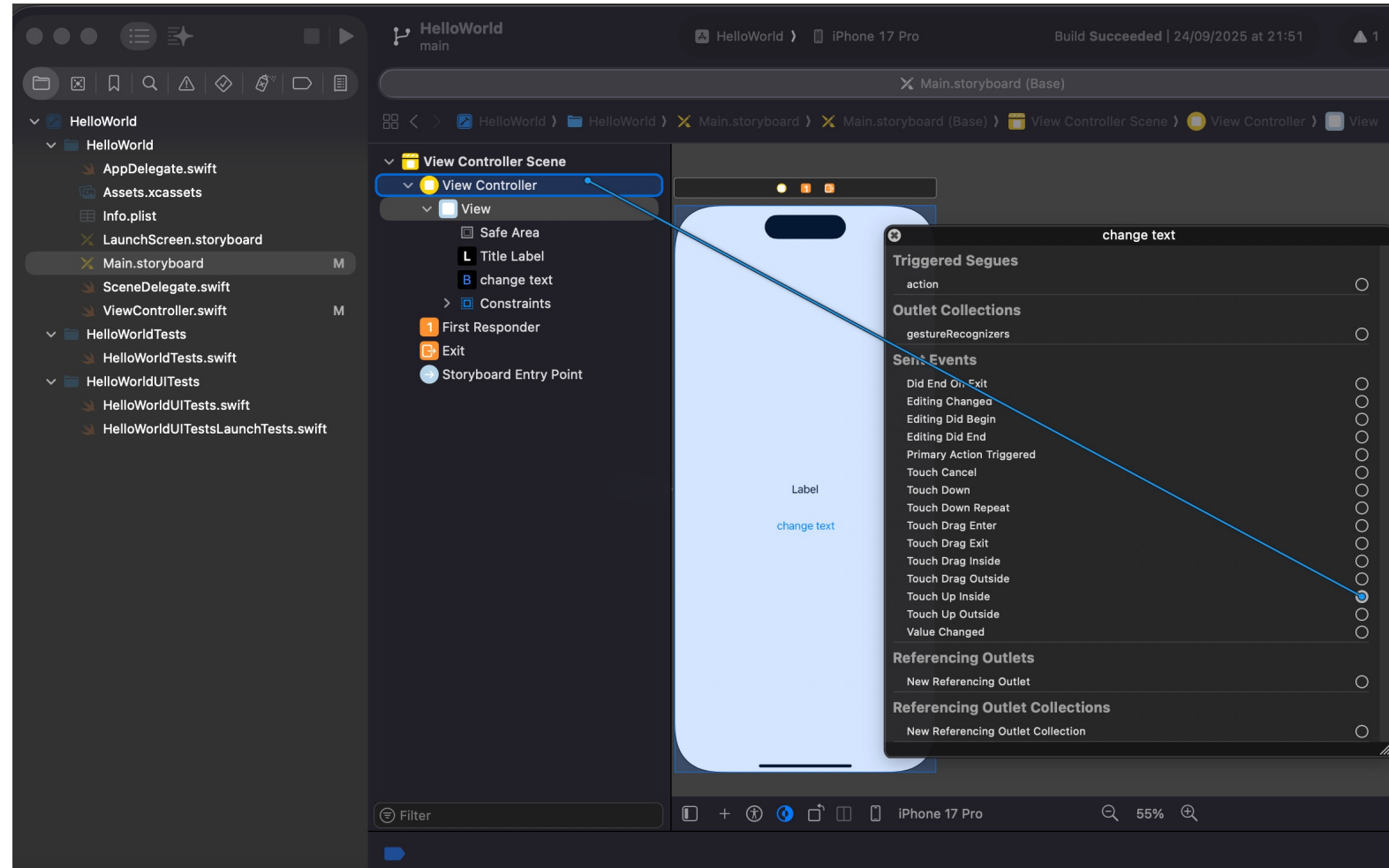


Hands-on

Criar primeira app em iOS

- A conexão Touch Up Inside significa que o utilizador tem de clicar dentro do objeto Botão.
- A ação de arrastar a conexão até ao View Controller liga o evento Touch Up Inside ao objeto View Controller fazendo com que este objeto seja notificado sempre que o botão for clicado

Arraste desde o círculo que identifica a conexão Touch Up Inside até ao ícone View Controller

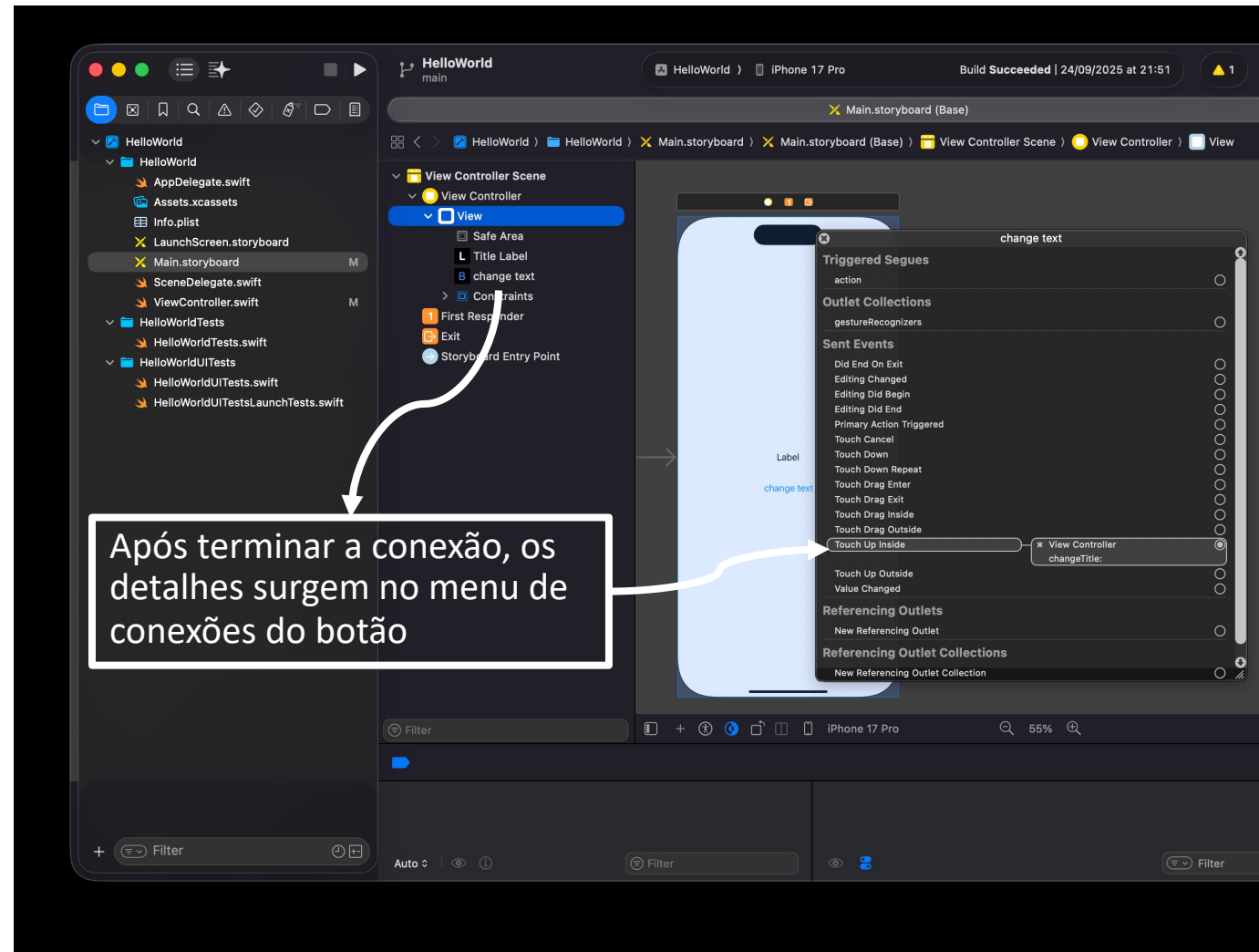


Hands-on

Criar primeira app em iOS

Assim que terminar a ação de arrastar surge uma lista com os métodos que podem ser utilizados na conexão.

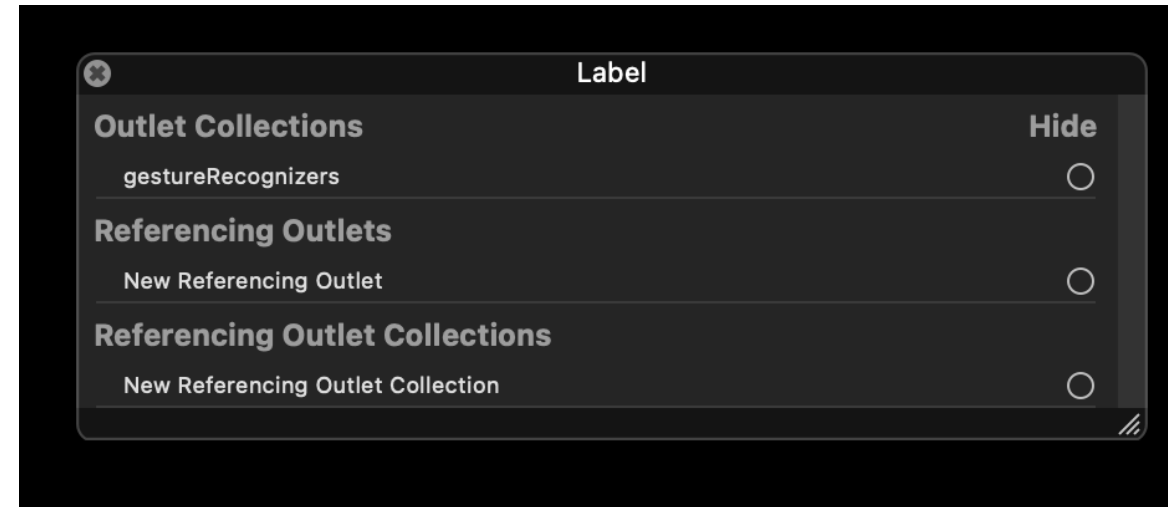
- Neste exemplo apenas existe um método possível `changeTitle`. Após selecionar este método o evento e o objeto ficam conectados



Hands-on

Criar primeira app em iOS

- Criação de uma conexão para o objeto Label
 - Não há preocupação com eventos sobre a Label;
 - Pretende-se conectar a variável (outlet) `nameLabel` do `ViewController` ao objeto inserido na interface do iPhone
 - Esta conexão indica qual o objeto que se encontra na interface do iPhone e no qual se pretende inserir o texto
- Clicar com o botão direito no objeto Label da interface
 - Não existem tantas opções como no objeto Button

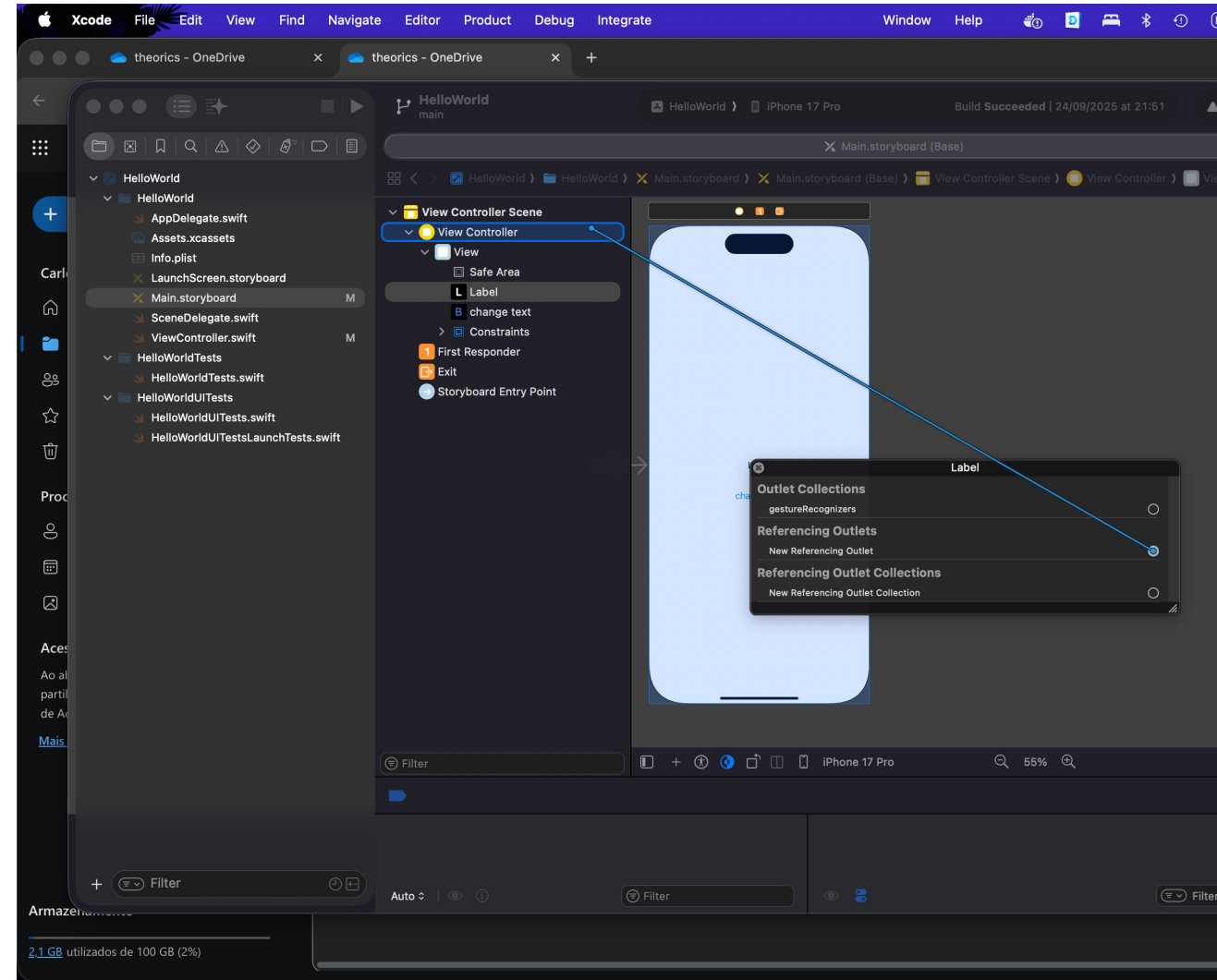


Hands-on

Criar primeira app em iOS

Criação de um referencing outlet

- Conecta o objeto do ecrã (interface) a uma variável no objeto ViewController
- Arrastar a opção New referencing outlet até ao ícone View Controller

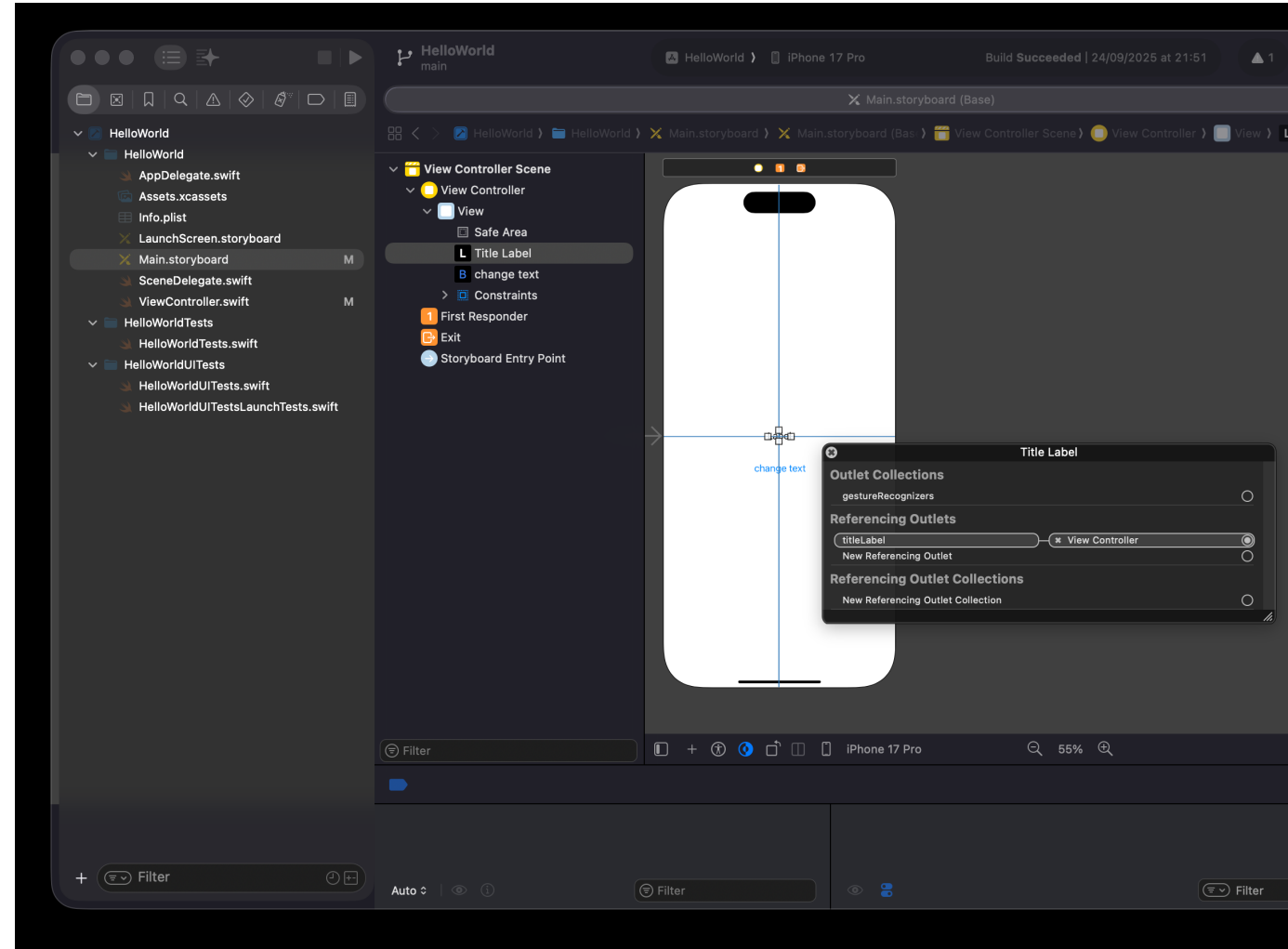


Hands-on

Criar primeira app em iOS

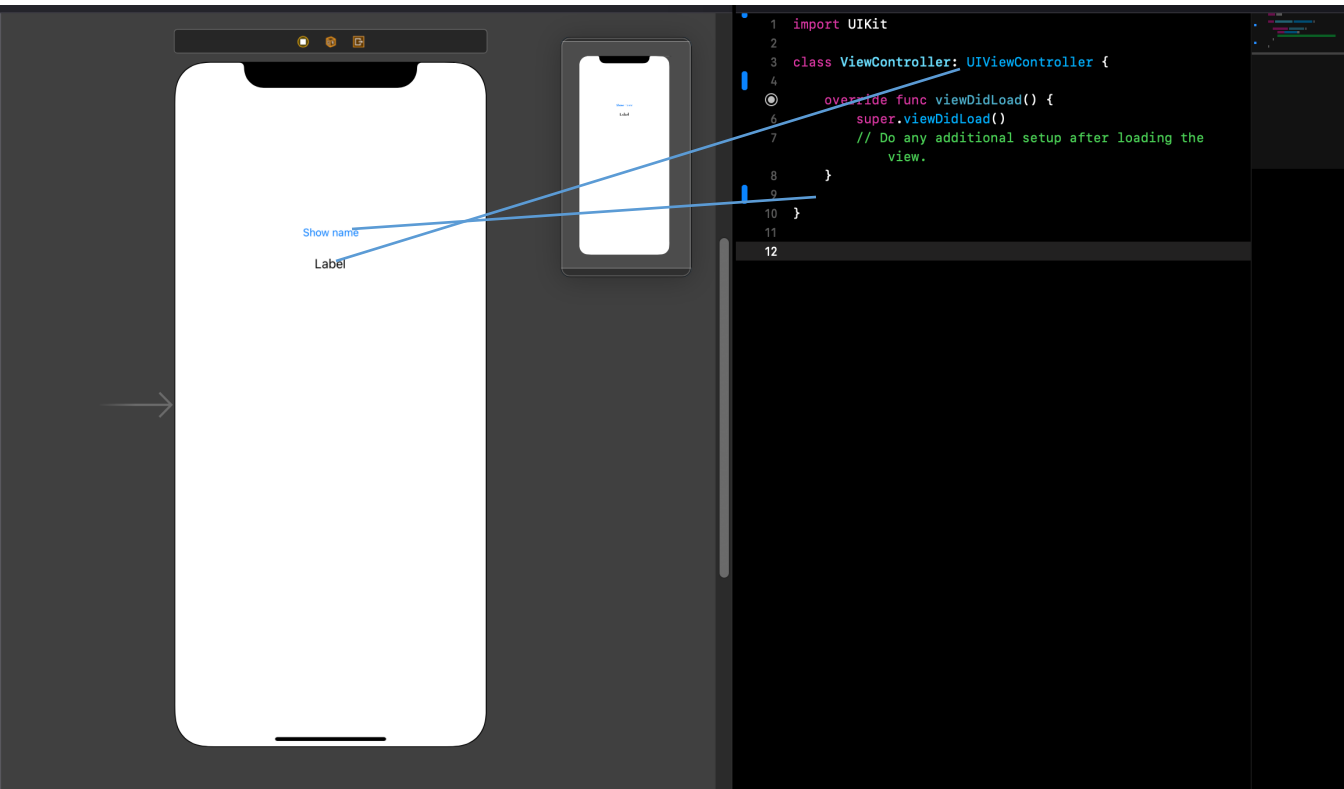
Quando terminado a ação de arrastar surge uma lista de outlets no objeto ViewController

- Selecionar a opção *titleLabel* entre as duas possíveis (nome da variável definida no objeto ViewController)



Hands-on

Criar primeira app em iOS



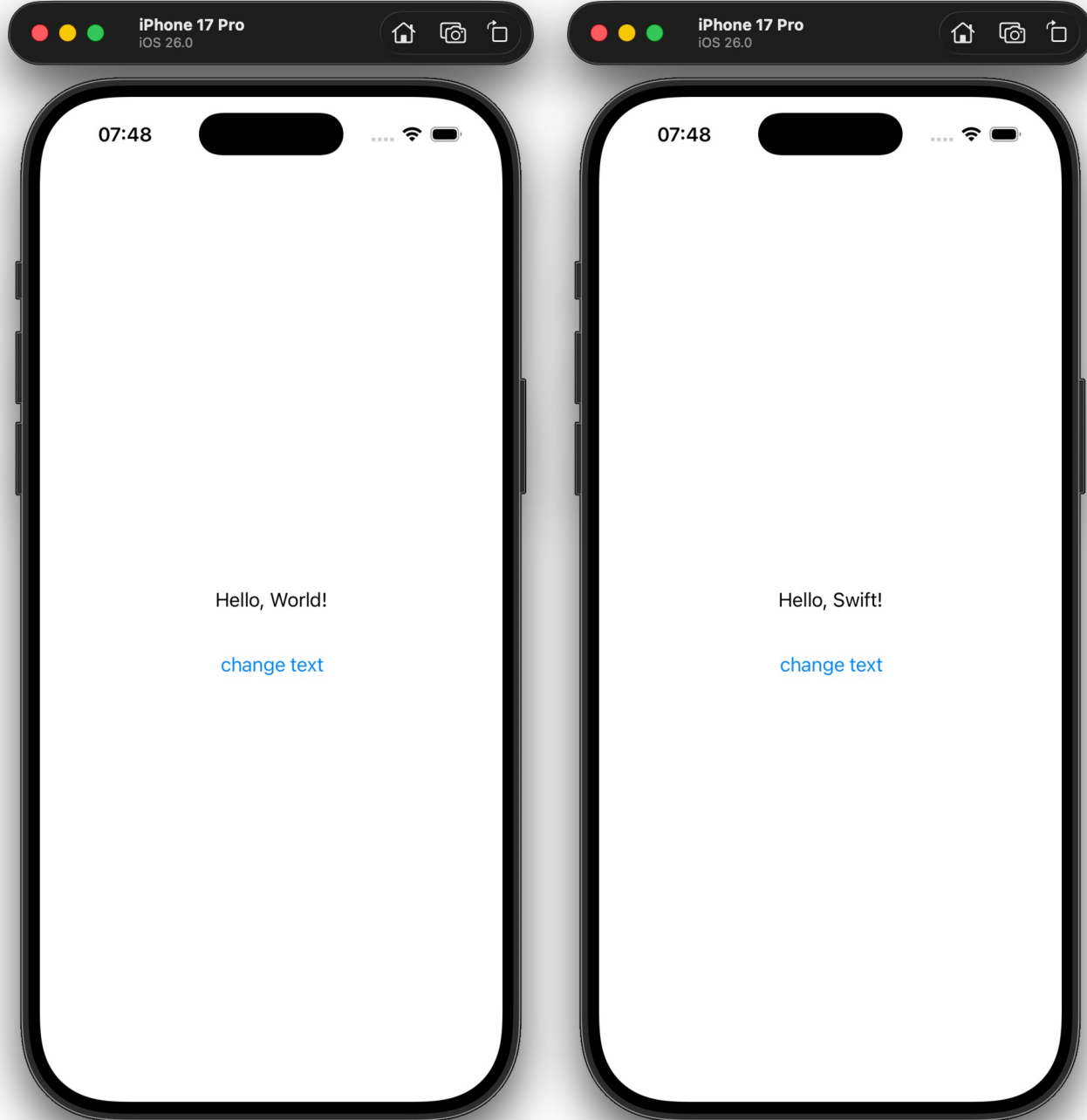
Em alternativa:

- Visualizar lado a lado o storyboard e o ViewController (use a tecla '*option*')
- Arraste com o botão direito do rato:
 - A *label* para a linha inicial do ViewController
 - O button para uma linha dentro do ViewController

Hands-on

Criar primeira app em iOS

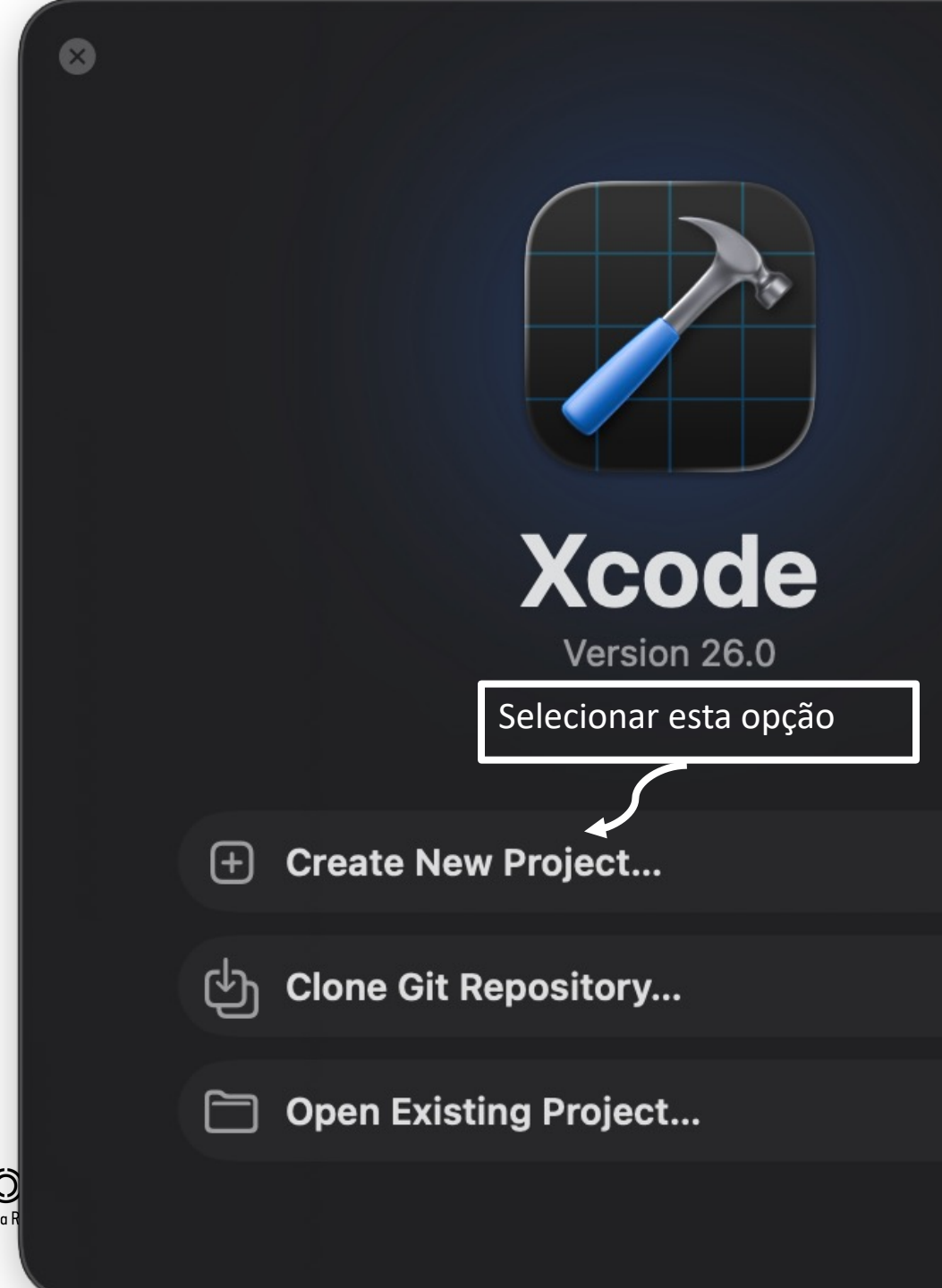
- Execução da aplicação
 - Clicar no botão play no canto superior esquerdo
 - Este processo grava automaticamente o código e executa um simulador de iPhone
- Quando clicar no botão *ChangeText*, o texto da Label irá mudar do seu valor por omissão na Label para o valor definido no código!
 - Nota: já é possível alterar na interface do iPhone (no Xcode) o valor por omissão da Label para uma string vazia



Hands on (parte 2)

Estações de rádio (RadioStations)

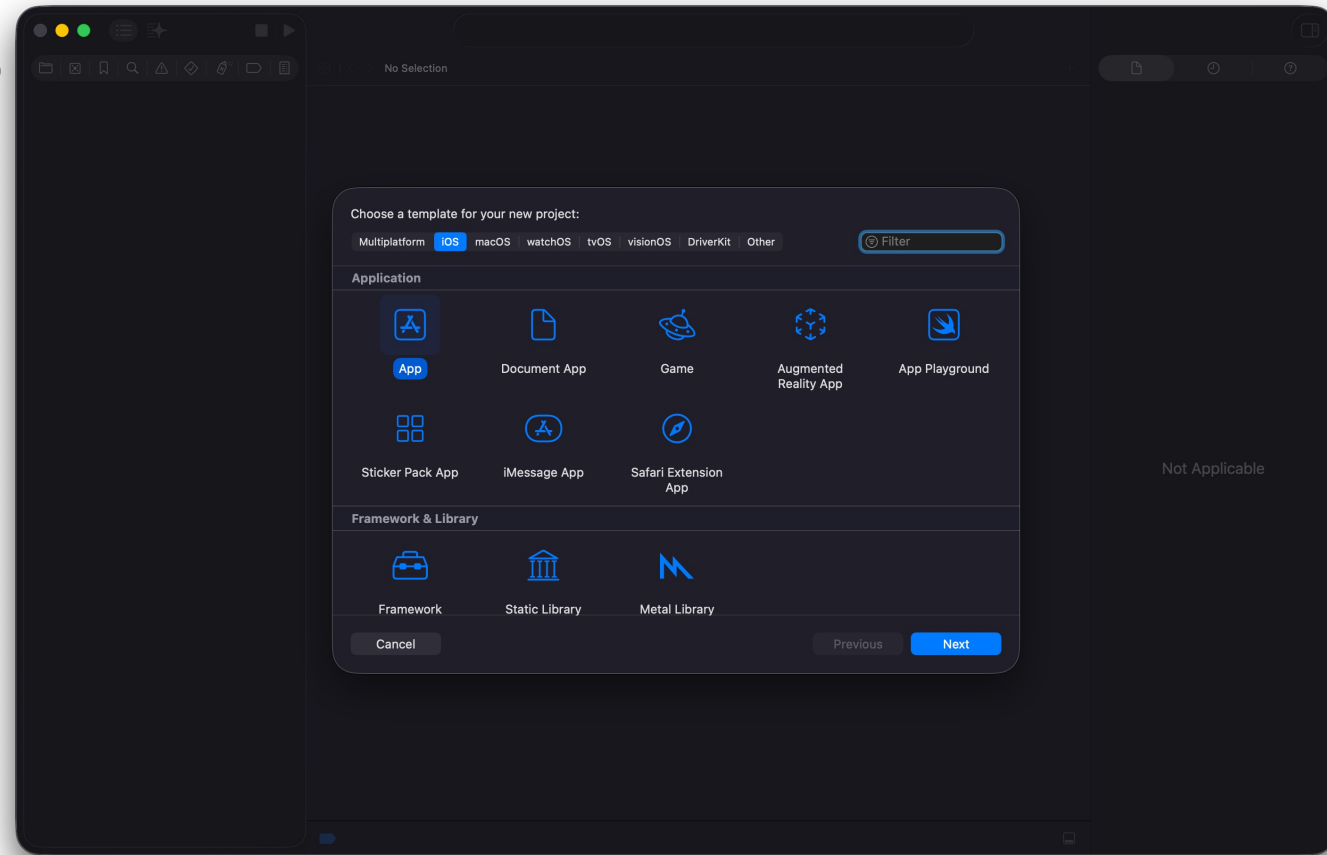
1. Crie um novo projeto Xcode (Create a new Xcode Project);



Hands on (parte 2)

Estações de rádio (RadioStations)

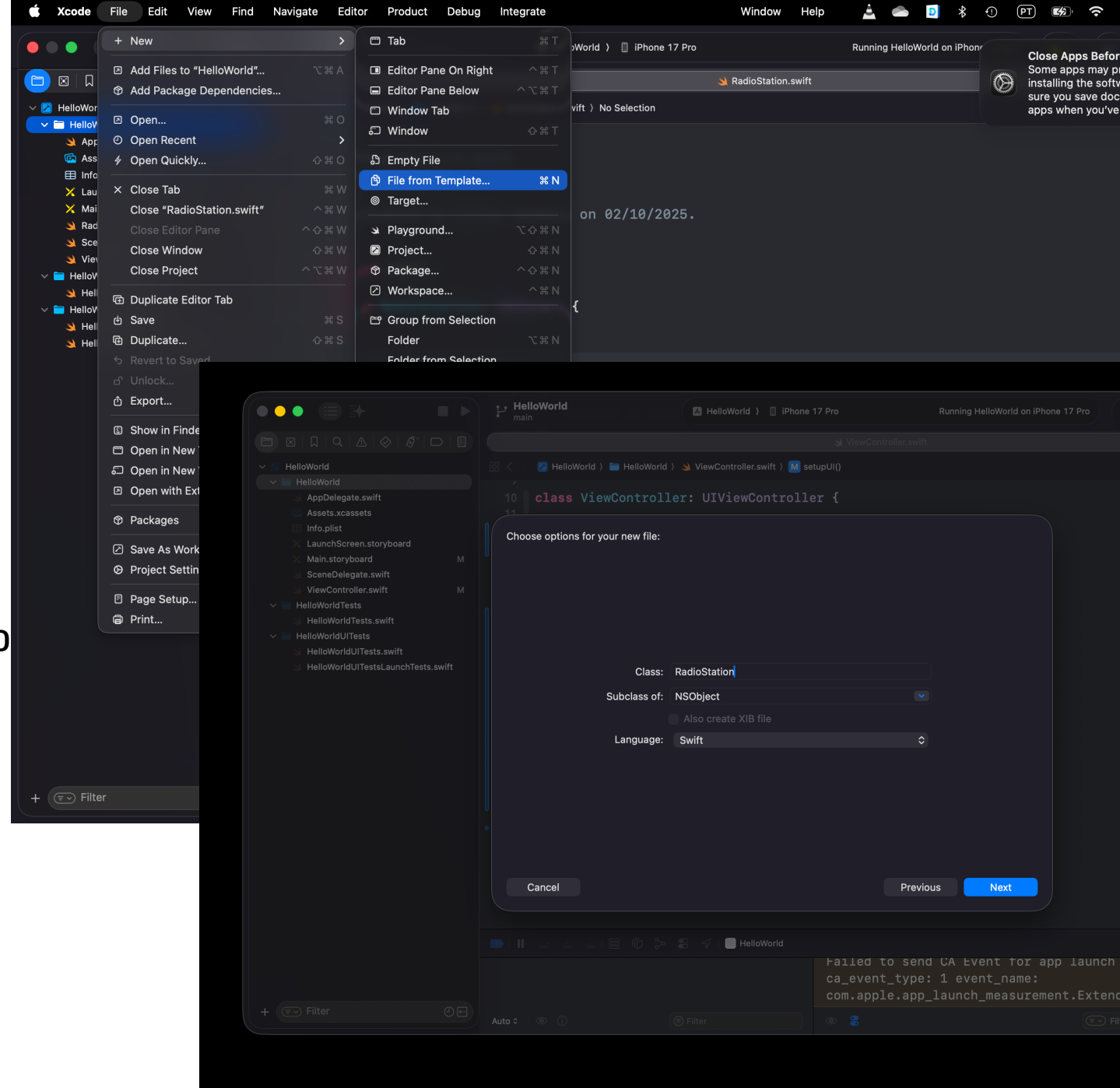
2. Selecione Single View Application dentro do menu iOS
3. Nome da aplicação: **RadioStations**



Hands on (parte 2)

Adição/criação de objetos

1. Criação do objeto RadioStation
 1. Selecionar a pasta RadioStations
 2. No menu File -> New
 3. Selecionar File From Template
 4. Escolher Cocoa Touch Class inserido grupo Source
 5. Inserir RadioStation como nome de classe e indicar que é subclasse de NSObject



Hands on (parte 2)

Implementação da classe

Após criação do ficheiro da classe é necessário escrever os atributos/propriedades e métodos

RadioStation
Name
Frequency
Min AM Frequency
Max AM Frequency
Min FM Frequency
Max FM Frequency
Is Band FM

```
9  import UIKit
10
11  class RadioStation: NSObject {
12
13      var name:String
14      var frequency:Double
15
16      override init() {
17          name = "Default"
18          frequency = 100
19      }
20
21      static var minAMFrequency:Double = 520.0
22
23      static var maxAMFrequency:Double = 1610.0
24
25      static var minFMFrequency:Double = 88.3
26
27      static var maxFMFrequency:Double = 107.9
28
29      func isBandFM() -> Int {
30          if frequency >= RadioStation.minFMFrequency &&
31             frequency <= RadioStation.maxFMFrequency {
32              return 1
33          }else{
34              return 0
35          }
36      }
37
38  }
```

Hands on (parte 2)

Criação da interface

Abra o ficheiro `ViewController.swift` de modo a definir variáveis para utilização nesta classe

- `@IBOutlet weak var stationName:UILabel!`
 - `@IBOutlet weak var stationFrequency:UILabel!`
 - `@IBOutlet weak var stationBand:UILabel!`
-
- Estas variáveis serão utilizadas para conectar a interface a elementos que se possam utilizar no código de modo a editar os valores a apresentar ao utilizador

```
11 class ViewController: UIViewController {  
12  
    @IBOutlet weak var stationName:UILabel!  
    @IBOutlet weak var stationFrequency:UILabel!  
    @IBOutlet weak var stationBand:UILabel!
```

Hands on (parte 2)

Criação da interface

Introduzir uma nova variável `myStation` do tipo `RadioStation`

- Tal como outra POO, não é obrigatório inicializar um objeto, porém é boa prática criar um inicializador de modo a inserir valores por omissão

```
21     var myStation:RadioStation
22
23     required init?(coder aDecoder: NSCoder) {
24         myStation = RadioStation()
25         myStation.frequency = 92.9
26         myStation.name = "Comercial"
27         super.init(coder: aDecoder)
28     }
```

Construtor

Valores por omissão

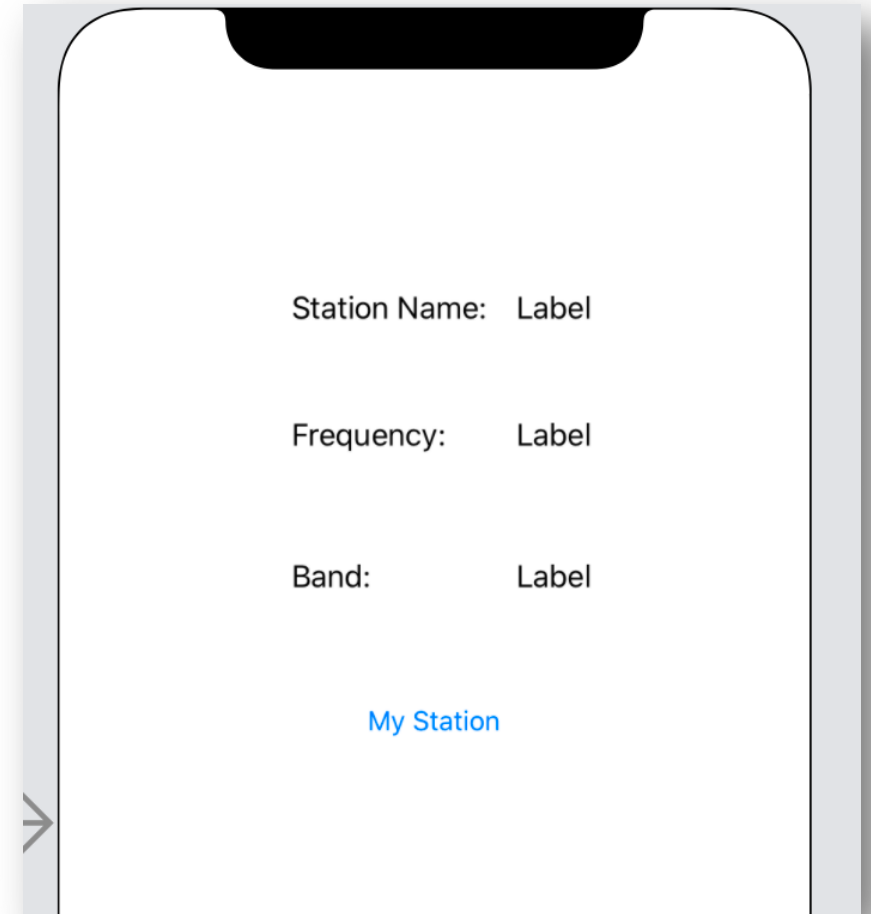
Hands on (parte 2)

Criação da interface

Criação das conexões entre os objetos da interface do iPhone e os objetos/variáveis declarados no objeto `ViewController.swift`

1. Abrir o ficheiro `Main.storyboard`
2. Adicionar 6 Labels e 1 Button

Nota: O botão, após ser clicado, irá provocar uma atualização da informação da estação de rádio

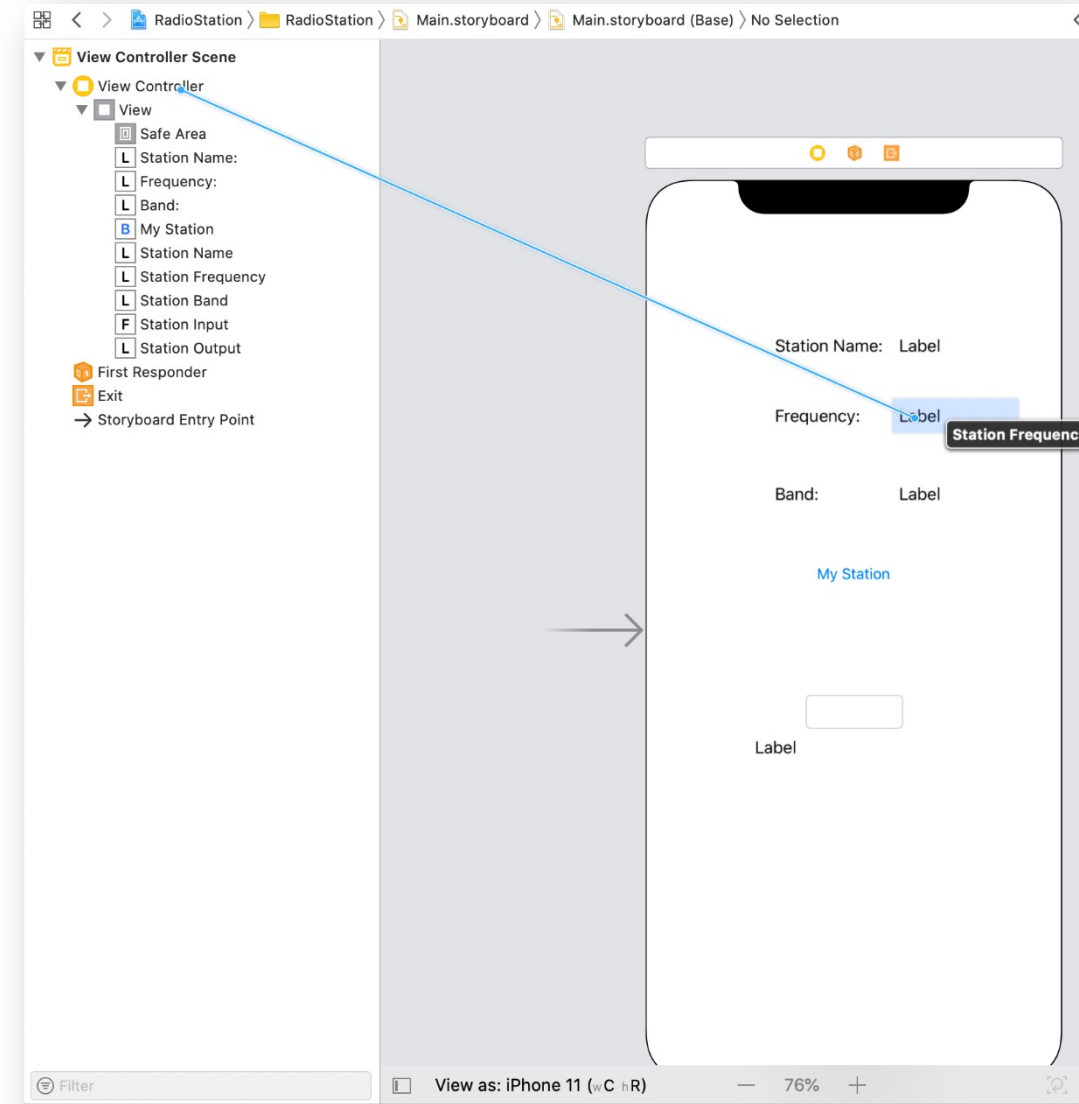


Hands on (parte 2)

Ligação entre os elementos

Com os objetos da interface posicionados é possível estabelecer as conexões entre interface e variáveis do programa

1. Segurando o botão Ctrl arraste o objeto ViewController até à Label que se encontra à direita da Station Name de modo a surgir a lista de outlets
 1. Neste caso será a propriedade stationName
2. Selecione o nome da propriedade que pretende apresentar nesse objeto Label
 1. Neste caso será a propriedade stationName
3. Repita o processo para as restantes Labels

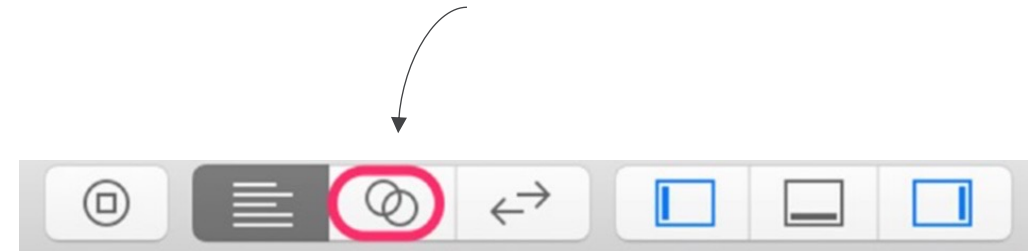


Hands on (parte 2)

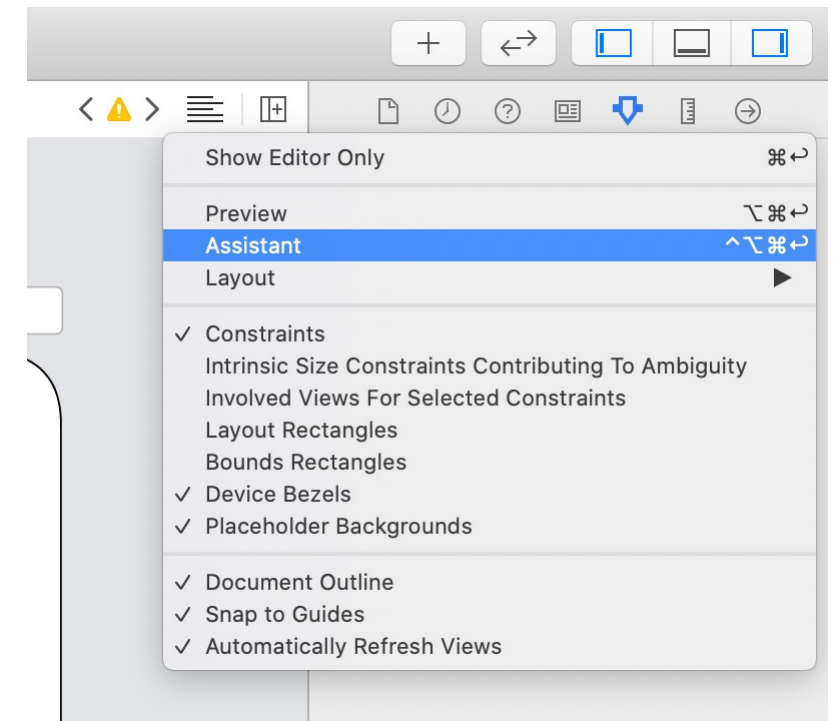
Ligação entre os elementos

- A conexão para o botão **obriga** a existência de um método no ViewController
- Este processo pode ser feito
 - Diretamente no código adicionando métodos do tipo @IBAction
 - Método de arrastar que facilita este processo

Abrir o Assistant Editor



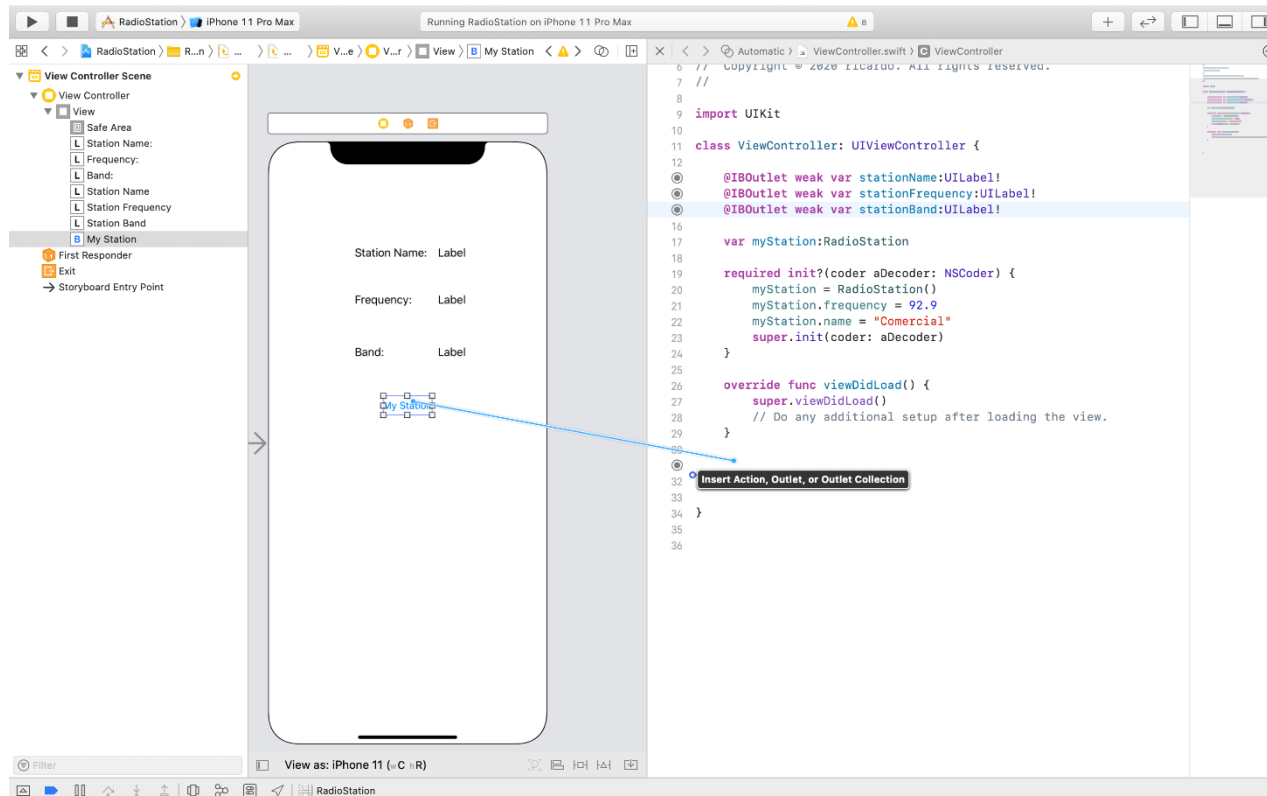
Ou alternativamente



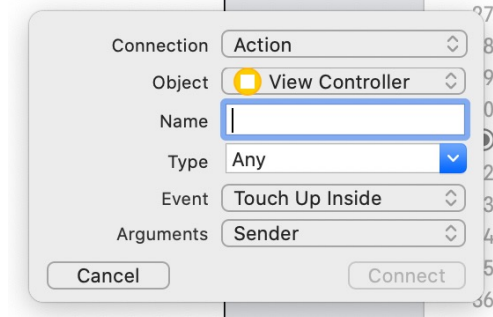
Hands on (parte 2)

Ligação entre os elementos

- Uma nova janela é aberta onde mostra o conteúdo do ViewController
- Com a tecla Ctrl pressionada arraste o botão da interface para a nova janela com o código



Quando deixar de pressionar o rato surge uma pequena janela



Selecione Action e insira o nome buttonClick

Hands on (parte 2)

Ligação entre os elementos

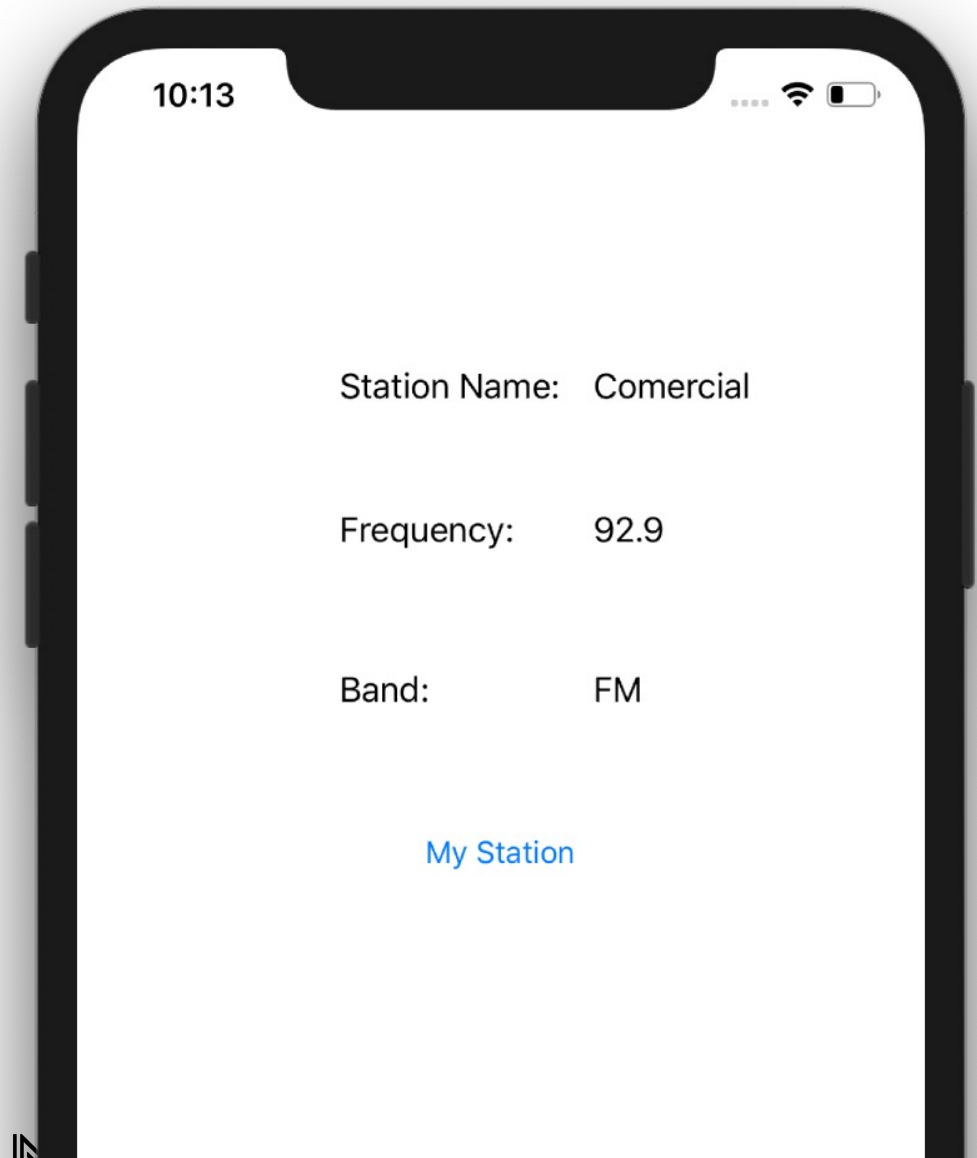
- O Xcode cria automaticamente o método
- Insira o código que implemente a ação de clicar no botão

```
@IBAction func buttonClick(_ sender: Any) {  
    stationName.text = myStation.name  
    stationFrequency.text = "\(myStation.frequency)"  
    if myStation.isBandFM() == 1 {  
        stationBand.text = "FM"  
    }else{  
        stationBand.text = "AM"  
    }  
}
```

Hands on (parte 2)

Execução da aplicação

- Executando a aplicação e pressionando o botão deverá obter um resultado idêntico à figura



Programação Para Dispositivos Móveis II

PRIMEIRA APLICAÇÃO iOS

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jmr@estg.ipp.pt, Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt